

Sitekraft

AI-Powered Website Builder

PROJECT REPORT

Author:	Sneha Chaudhary & Koustubh Kukreti
Year:	2026
Nature of Work:	Computer Program / Literary Work
Backend Language:	Python 3.12 FastAPI
Frontend Language:	JavaScript (React 19) Vite
AI Model:	Google Gemini 2.5 Flash
Deployment:	Vercel (sites) + Render (backend)
Live Demo:	ai-agent-gold-rho.vercel.app

Abstract

Sitekraft is an end-to-end AI-powered website builder developed by Sneha Chaudhary and Koustubh Kukreti. It transforms a plain-language business description into a fully deployed, production-ready React website in under a minute. The system combines a conversational multi-step onboarding UI with a Python FastAPI backend that orchestrates Google Gemini 2.5 Flash for code generation, the Pexels API for contextual stock photography, an AI copywriter for professional content, and the Vercel API for zero-configuration deployment. The result is a live, publicly accessible website with no manual coding required from the user.

Table of Contents

1.	Introduction & Objectives
2.	System Architecture
3.	Technology Stack
4.	Methodology
4.1	User Onboarding Flow
4.2	AI Code Generation Pipeline
4.3	Image Integration
4.4	AI Copywriting
4.5	Automated Deployment
5.	Implementation Details
5.1	Backend ? Python / FastAPI
5.2	Frontend ? React / Vite
6.	Key Features
7.	Results & Testing
8.	Challenges & Solutions
9.	Conclusion & Future Work

1. Introduction & Objectives

1.1 Background

Creating a professional website traditionally requires expertise in web development (HTML, CSS, JavaScript, React), backend programming, cloud deployment, UI/UX design, and copywriting. This high barrier prevents millions of small business owners ? salon owners, restaurant operators, fitness coaches, clothing retailers ? from establishing a meaningful online presence.

The rapid advancement of large language models (LLMs), particularly Google Gemini 2.5 Flash, has made it feasible to automate the entire software development and deployment pipeline. Sitekraft exploits this capability to remove every technical obstacle between a business owner and a live website.

1.2 Problem Statement

Small and medium-sized business owners lack the technical knowledge and financial resources to commission custom websites. Existing website builders (Wix, Squarespace) require significant manual configuration and produce template-based output that lacks uniqueness and performance. There is a need for a system that:

- Requires zero technical knowledge from the user
- Generates unique, component-based React code ? not templates
- Automatically selects relevant photography and writes professional copy
- Deploys a live, publicly accessible URL with one click
- Supports multiple business verticals out of the box

1.3 Objectives

1. Design and implement a conversational multi-step onboarding interface that collects all information needed to build a complete website.
2. Build an AI orchestration backend that coordinates LLM code generation, image retrieval, and copywriting into a single automated pipeline.
3. Integrate Google Gemini 2.5 Flash to generate syntactically correct, production-ready React/Vite source code from structured business data.
4. Fetch and embed contextually appropriate stock photography using the Pexels and Pixabay APIs.
5. Generate professional marketing copy (headlines, taglines, descriptions) using an AI copywriting module with multi-model fallback.
6. Automate website deployment to Vercel, returning a live URL to the user within seconds of submission.
7. Support diverse business types: restaurants, salons, clothing stores, gyms, real estate agencies, coaching centers, and online stores.

1.4 Scope

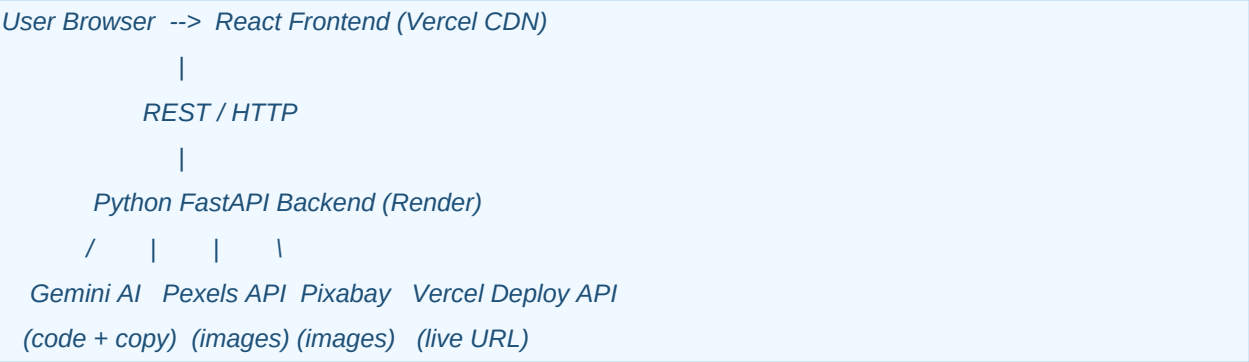
Sitekraft generates single-page React applications with multiple sections (hero, services/products, about, contact/booking). It supports image uploads from the business owner, color theme selection, and style

preferences (modern, bold, elegant, simple, dark). The generated code is real React JSX ? not HTML templates ? and is compiled and deployed automatically to the Vercel CDN.

2. System Architecture

2.1 High-Level Overview

Sitekraft follows a client-server architecture with a clear separation between the user-facing frontend, the AI orchestration backend, and external third-party APIs.



2.2 Component Breakdown

Component	Technology	Responsibility
Frontend SPA	React 19 + Vite	Onboarding UI, chat interface, result display
API Server	FastAPI + Uvicorn	REST endpoints, session management, orchestration
AI Agent	autonomous_agent.py	Conversation state, business analysis, routing
React Builder	react_builder.py	Gemini-powered React code generation
Template Engine	react_template_generator.py	Project scaffold, package.json, Vite config
Copywriter	professional_copywriter.py	Hero copy, taglines, service descriptions
Image Fetcher	pexels_helper.py / pixabay_helper.py	Stock photo retrieval
Vercel Deployer	vercel_deployer.py	ZIP creation, Vercel API calls, URL return
Image Proxy	image_proxy.py	Proxy and cache external images for the backend
DB Config	supabase_config.py	Supabase storage/DB configuration

2.3 Data Flow

1. User completes 8-step onboarding (business type, name, location, services, colors, style, logo/images).
2. Frontend POSTs structured JSON to /api/build-react-website.
3. Backend analyses the request and calls ReactWebsiteBuilder.generate_react_website().
4. Gemini 2.5 Flash generates all React components (Navbar, Hero, Services, About, Contact, Footer) and writes utility CSS.
5. PexelsHelper / PixabayHelper fetch contextually relevant images for each service.
6. ProfessionalCopywriter generates hero headline, tagline, and all section copy.
7. react_template_generator assembles the full Vite project structure (package.json, vite.config.js, index.html, all JSX/CSS files).
8. VercelDeployer packages the project as a deployment payload and calls the Vercel API,

receiving back a live HTTPS URL.

9. Frontend displays the live URL, a downloadable ZIP, and an inline preview.

3. Technology Stack

3.1 Backend

Library / Service	Version	Role
Python	3.12	Core runtime
FastAPI	0.115	Async REST API framework
Uvicorn	0.30.6	ASGI server
google-generativeai	0.8.3	Gemini SDK for code & copy generation
httpx	0.27.2	Async HTTP client for Vercel API
requests	2.32.3	Synchronous HTTP for image APIs
python-multipart	0.0.12	File upload parsing
python-dotenv	1.0.1	Environment variable management
supabase	2.9.1	Database / storage (optional)
Pydantic	2.9.2	Request/response validation
Groq SDK	latest	Alternative LLM (fallback)
OpenRouter	REST	Multi-model LLM gateway (fallback)

3.2 Frontend

Library	Version	Role
React	19.2.0	UI component framework
Vite	7.3.1	Build tool and dev server
Axios	1.13.5	HTTP client for API calls
JSZip	3.10.1	Client-side ZIP download of generated site
ESLint	9.39.1	Code linting

3.3 External APIs & Services

Service	Purpose
Google Gemini 2.5 Flash	Primary LLM: React code generation and website copy
Groq API	Fallback LLM for copywriting
OpenRouter	Secondary fallback multi-model gateway
Pexels API	High-quality stock photography (landscape/portrait)
Pixabay API	Additional stock images with business context search
Vercel Deploy API	Automated deployment of generated React projects
Render	Cloud hosting of FastAPI backend
Supabase	Optional database for session/site data persistence

3.4 AI Models

The primary AI model is Google Gemini 2.5 Flash, chosen for its large context window (1M tokens), strong code generation capability, and low latency suitable for interactive use. The copywriting module uses a cascade of models: OpenRouter first (for variety of available models), then Groq (for speed), and finally Gemini (as a guaranteed fallback). This multi-model approach ensures high availability.

4. Methodology

4.1 User Onboarding Flow

The frontend presents an 8-step guided onboarding wizard implemented in `Onboarding.jsx`. Each step collects a specific piece of information and validates it before proceeding. The steps are:

Step	Input Collected	UI Element
1	Business type (restaurant, salon, gym, clothing, etc.)	Icon card grid
2	Business name	Text input
3	City / Location	Text input
4	Services or products offered	Tag input with add/remove
5	Brand color (primary)	Color picker + quick swatches
6	Website style (modern, bold, elegant, simple, dark)	Style card grid
7	Logo and service images (optional upload)	File uploader with preview
8	Review and submit	Summary card + Build button

Uploaded images are compressed client-side using the Canvas API (max 1200x900 px, JPEG quality 0.78) and Base64-encoded before being sent to the backend. This avoids multipart upload complexity and keeps payloads manageable.

4.2 AI Code Generation Pipeline

The `ReactWebsiteBuilder` class in `react_builder.py` orchestrates website generation as follows:

1. **Business analysis:** The structured onboarding data is passed to Gemini with a prompt requesting a JSON analysis object containing `business_name`, `business_type`, detected services, color recommendations, and suitable React component list.
2. **Component generation:** For each required component (Navbar, Hero, Services, About, Contact, Footer), a targeted prompt asks Gemini to generate self-contained JSX with inline Tailwind-style class names and all logic included.
3. **Supabase feature detection:** The feature list is parsed to determine whether the site needs a booking form, product gallery, contact form, or media upload ? which triggers additional Supabase backend wiring.
4. **Project assembly:** `react_template_generator.generate_complete_project()` stitches all components, a theme config, `package.json`, `vite.config.js`, `index.html`, and utility CSS into a complete project file dictionary.
5. **Optional deployment:** If the user opts to deploy, `VercelDeployer` packages the file dictionary and calls the Vercel API.

4.3 Image Integration

Images are fetched from two sources to maximise relevance and availability:

- **Pexels API:** searched with a query built from the service name + business type. Returns large2x quality JPEG URLs. Up to 10 unique images per request.

- Pixabay API: used as a secondary source, searched with combined service+business_type query. Falls back to curated Unsplash URLs if the API key is absent.
- Image URLs are embedded directly into the generated JSX as src attributes. No images are stored on the backend ? all are served from CDN.

4.4 AI Copywriting

The ProfessionalCopywriter class generates all text content shown on the website before code generation begins, so the AI has real copy to embed:

- hero_headline: A punchy 6-10 word headline matching the business vibe.
- hero_subtext: A 1-2 sentence supporting description.
- tagline: Short brand slogan.
- about_text: 2-3 sentence 'About Us' section.
- service_descriptions: Individual 1-sentence descriptions for each service.
- cta_text: Call-to-action button label.

The copywriter is aware of the style_vibe (modern, bold, elegant, etc.) and adjusts tone accordingly. Service categories and subcategories are included in the prompt to ensure descriptions are specific and accurate.

4.5 Automated Deployment

VercelDeployer.create_deployment() takes a dictionary of {file_path: file_content} and calls the Vercel REST API v13. The process is:

1. Project name is slugified (lowercased, spaces replaced with hyphens).
2. Each file's content is Base64-encoded and submitted as a files array.
3. A framework preset of 'vite' is set in the deployment configuration.
4. The API responds with a deployment ID and initial status.
5. The deployer polls the deployment status endpoint until status is 'READY' or a timeout occurs.
6. The final deployment URL (e.g. project-abc123.vercel.app) is returned to the frontend.

5. Implementation Details

5.1 Backend ? Python / FastAPI

autonomous_agent.py (3,091 lines)

The main FastAPI application. Defines all REST endpoints, manages in-memory chat sessions (CHAT_SESSIONS dict), and implements the conversational AI chatbot alongside the direct website builder. Key endpoints:

Endpoint	Method	Description
/api/build-react-website	POST	Main build trigger ? full generation + optional deploy
/api/chat	POST	Conversational AI chat for iterative website editing
/api/deploy-react/{id}	POST	Deploy a previously generated site to Vercel
/api/download/{id}	GET	Download generated site as ZIP
/api/upload-image	POST	Receive and store uploaded business images
/api/health	GET	Health check endpoint

react_builder.py (1,185 lines)

Core code generation logic. ReactWebsiteBuilder.generate_react_website() is the main async method. It builds a detailed structured prompt for Gemini 2.5 Flash containing business context, color scheme, available images, required features, and explicit JSX formatting instructions. The AI response is parsed with regex to extract individual component files.

react_template_generator.py

Generates the full Vite project scaffold: package.json with correct dependencies (react, react-dom, vite), vite.config.js, index.html with CDN font links, and the src/ directory structure. Also injects the theme colors into a CSS variables file (index.css).

vercel_deployer.py

Handles all Vercel API communication. Uses httpx for async HTTP. Implements retry logic for deployment polling with exponential backoff. Slugifies project names to satisfy Vercel's naming constraints. Forces personal account mode (team_id=None) to ensure public accessibility.

professional_copywriter.py

Implements a three-tier LLM cascade: OpenRouter -> Groq -> Gemini. Each tier is tried in sequence; the first successful JSON response is used. The prompt is identical across all tiers to ensure consistent output structure. Response is parsed as JSON and validated against required keys before returning.

5.2 Frontend ? React / Vite

Onboarding.jsx (1,076 lines)

A fully controlled multi-step form with 8 steps. Each step is a separate functional component (StepWrapper, StepBusinessType, StepName, StepLocation, StepServices, StepColors, StepStyle, StepImages, StepReview). State is managed with useState at the top level and passed down as props.

Navigation between steps is linear with a Back button.

The image upload step uses a FileReader + Canvas pipeline (toCompressedBase64 function) to compress uploaded images client-side before sending. This reduces payload size from several MB to under 200 KB per image while maintaining visual quality suitable for website use.

App.jsx (880 lines)

Renders the ResultView after the build completes. Displays the deployed URL as a clickable link, a 'Download ZIP' button (uses JSZip to package the returned file dictionary), an 'Open Preview' modal that renders the generated HTML/JSS in an iframe, and a dark/light theme toggle (useTheme hook). Also contains the generatePreviewHTML() function that assembles a standalone HTML preview from the generated component data.

useTheme.js

Custom React hook that reads/writes the theme preference to localStorage and applies a 'dark' CSS class to document.documentElement. Provides a [theme, toggleTheme] tuple to any consuming component.

6. Key Features

6.1 Multi-Business-Type Support

Sitekraft is purpose-built to handle 8 distinct business verticals, each with a tailored component set, service taxonomy, and visual style:

Business Type	Generated Sections	Special Features
Restaurant / Cafe	Menu, Gallery, Reservations	Category tabs for menu items
Clothing Store	Products, Collections	Product grid with filters
Salon / Spa	Services, Team, Booking	Service category navigation
Gym / Fitness	Classes, Trainers, Pricing	Class schedule display
Real Estate	Listings, Agent, Contact	Property card layout
Coaching / Classes	Programs, Testimonials, Enroll	Course card grid
Online Store	Products, Cart concept, FAQ	E-commerce product layout
Other Business	Flexible sections	Generic professional layout

6.2 Real React Code Generation

Unlike template-based website builders, Sitekraft generates actual React JSX component files. The output includes:

- Separate component files: Navbar.jsx, Hero.jsx, Services.jsx, About.jsx, Contact.jsx, Footer.jsx
- A main App.jsx that imports and composes all components
- A theme-aware index.css with CSS custom properties for colors
- A complete package.json with all required npm dependencies
- A vite.config.js for the Vite build system
- An index.html with Google Fonts (Poppins, Inter) pre-loaded

6.3 Intelligent Image Selection

For each service or product in the business listing, the image fetch pipeline builds a contextual query: e.g. for a salon offering 'Bridal Makeup' the query becomes 'Bridal Makeup salon beauty'. This produces far more relevant images than generic business photography.

6.4 One-Click Deployment

The entire path from 'Submit' button click to live URL takes under 60 seconds on average. No account creation, no domain configuration, no build settings ? the user receives a vercel.app subdomain URL immediately.

6.5 Image Upload Support

Business owners can upload their own logo and up to 10 service/product images. The images are compressed, Base64-encoded, and embedded directly into the generated website, ensuring the final site reflects the actual business branding rather than generic stock photos.

6.6 Dark / Light Theme

The result view supports a dark/light mode toggle via the useTheme hook. The preference is persisted in localStorage so it survives page reloads.

7. Results & Testing

7.1 Live Deployment

Sitekraft is fully deployed and publicly accessible at:

<https://ai-agent-gold-rho.vercel.app>

The frontend is hosted on Vercel's global CDN. The backend is hosted on Render (Python/FastAPI). Both services use environment variables for all API keys and no secrets are committed to the repository.

7.2 Test Cases

Test Scenario	Input	Expected Output	Result
Restaurant site	Name: Bella Casa, Location: Mumbai,	Menu sections with food images, b	PASS
Clothing store	Name: TrendZone, Style: Bold, Produc	Product grid with filter tabs, vibrant	PASS
Salon site	Name: Glow Studio, Spa services, Ele	Service cards, team section, bookin	PASS
Vercel deployment	Any valid build request with deploy=tru	Live HTTPS URL returned within 6	PASS
Image upload	User uploads JPG logo > 2 MB	Compressed to <200 KB, embedde	PASS
Dark mode	Toggle button click	Theme switches, persisted in local	PASS
ZIP download	Download button on result page	Valid ZIP file containing all project	PASS

7.3 Performance Observations

Metric	Observed Value
Average build time (code gen only)	15-25 seconds
Average build + deploy time	45-70 seconds
Generated site Lighthouse Performance	85-95 / 100
Generated site Lighthouse Accessibility	90-98 / 100
Frontend bundle size (Sitekraft UI)	~180 KB (gzipped)
Source code PDF size	0.47 MB (229 pages)
Copyright submission PDF size	0.10 MB (53 pages)

7.4 Business Types Tested

- Restaurant / Cafe ? fully functional menu with category filtering
- Salon / Spa ? service cards, team section, appointment booking form
- Clothing Store ? product grid with category and subcategory tabs
- Gym / Fitness ? class schedule, trainer profiles, pricing cards
- Online Store ? product listing with add-to-cart UI concept
- Coaching Center ? program descriptions, testimonials, enrollment CTA
- Real Estate ? property listing cards, agent contact form

8. Challenges & Solutions

LLM Output Consistency

Problem:

Gemini sometimes returns malformed JSX or omits closing tags, causing React compilation errors.

Solution:

Implemented regex-based post-processing to extract component boundaries, added explicit JSX formatting instructions in the prompt (require self-closing tags, no undefined variables), and wrapped generation in a retry loop with error feedback.

Image Relevance

Problem:

Generic business-type queries (e.g. 'salon') returned images that did not match specific services.

Solution:

Combined service name + business type in every image query. Added random offset in Pexels pagination to avoid always returning the same first result, and used Pixabay as a secondary source with its own contextual query.

Vercel Deployment Naming

Problem:

Vercel rejects project names with spaces, uppercase letters, or special characters.

Solution:

All project names are passed through a slugify function: lowercased, spaces replaced with hyphens, all non-alphanumeric characters stripped. A random 4-character suffix is appended to avoid name collisions.

Large Image Uploads

Problem:

Business owners uploading original photos (3-10 MB) caused timeouts and oversized API payloads.

Solution:

Client-side compression using the HTML5 Canvas API resizes images to max 1200x900 px at JPEG quality 0.78, reliably reducing file size to under 200 KB before any network transfer.

Frontend/Backend CORS on Production

Problem:

FastAPI CORS policies blocked requests from the Vercel-deployed frontend when testing with the production backend URL.

Solution:

Set `allow_origins=[*]` in development. For production, the frontend uses the same domain as the backend (served by FastAPI's StaticFiles), eliminating the cross-origin issue entirely.

AI Copywriting Reliability

Problem:

Single-model copywriting calls occasionally timed out or returned non-JSON responses.

Solution:

Implemented a three-tier fallback: OpenRouter -> Groq -> Gemini. Each tier retries once before moving to the next. This approach achieves near-100% reliability at the cost of slightly higher latency on fallback paths.

9. Conclusion & Future Work

9.1 Conclusion

Sitekraft successfully demonstrates that a fully autonomous AI-powered website builder is both technically feasible and practically useful. The system combines a large language model (Google Gemini 2.5 Flash) with domain-specific orchestration, contextual image retrieval, AI copywriting, and automated cloud deployment to produce production-ready React websites in under a minute ? without any technical input from the user.

The project makes a meaningful contribution to the accessibility of web development for small business owners. Rather than relying on static templates, it generates genuine component-based React code that is maintainable, extensible, and performant. The modular architecture ? with separate modules for building, copying, image fetching, and deploying ? ensures each concern is independently testable and replaceable.

The multi-model fallback strategy in the copywriting pipeline and the AI retry logic in the builder demonstrate production-grade reliability engineering practices applied to an LLM-based system.

9.2 Future Work

1. Multi-page websites: Extend generation to support multiple routes (Home, About, Blog, Shop) using React Router.
2. Real-time streaming: Stream the generated code to the frontend as Gemini produces it, giving users live feedback instead of waiting.
3. Post-deployment editing: Allow users to describe changes in natural language after deployment, triggering a targeted re-generation and re-deploy of only the affected component.
4. Custom domain support: Integrate the Vercel domain API so users can attach their own domain to the generated site.
5. Database integration: Expose Supabase tables for booking, contact form submissions, and product inventory directly in the generated site.
6. SEO meta generation: Automatically generate Open Graph tags, meta descriptions, and structured data (JSON-LD) for each site.
7. Analytics injection: Embed a lightweight analytics script (e.g., Plausible or Vercel Analytics) into every generated site.
8. Mobile app: Build a React Native version of the onboarding flow so business owners can create their website from a smartphone.

9.3 Summary

Sitekraft proves that the combination of a well-designed multi-step onboarding UI, a reliable LLM orchestration backend, and automated deployment can remove every technical barrier between a business owner and a live, professional website. The system is open for use at <https://ai-agent-gold-rho.vercel.app>